

Designing Against Hold-Up: A Two-Axis Sovereignty Framework for Open-Source Business Models in Connected Hardware

An Embedded Case Study of a Smart-Lock Venture — Version 4.7

Truong Viet Phan (Vince Phan)

eBizco Australia Pty Ltd

August 2026

With architecture figures, an implementation-status appendix, refined CRA alignment (Annex III classification), license architecture, and a design-science framing of pre-market status.

Abstract

Connected hardware products routinely place consumers in a position transaction cost economics calls hold-up: once a customer has made a relationship-specific investment—installing a lock, wiring a hub—the vendor who controls the supporting cloud can unilaterally alter terms, degrade functionality, or withdraw support altogether. The smart-home industry’s two dominant governance models each solve one dimension of this problem while sacrificing the other: white-label ecosystems offer open market entry but route device traffic through vendor-controlled cloud infrastructure; interoperability standards such as Matter enable local control but gate market entry behind consortium membership and certification fees governed by incumbent firms.

This paper develops a two-axis framework—runtime sovereignty (control over data and functionality while a device operates) and admission sovereignty (control over which manufacturers may build for a given ecosystem)—to explain this pattern and to propose a structural alternative. We argue that combining open software with a services-based revenue model offers a defence against hold-up that is architectural rather than contractual, and we evaluate the framework through an embedded single-case study of a smart-lock venture in late-stage development. The case’s mechanisms—an owner-generated root key the vendor never holds, a broker designed to be unable to decrypt command content, a self-hosting option that gives customers a technical exit without vendor cooperation, and an open license architecture (MIT for protocol/SDK, GPLv3 for server/bridge/app, CERN-OHL-S for hardware)—are analyzed as varying-strength Williamsonian hostages.

The analysis yields four main findings. First, runtime sovereignty removes the vendor’s ex-post leverage by structurally forgoing the complementary asset that opportunism requires. Second, admission sovereignty removes the ex-ante barrier by publishing the protocol and reference implementation under open licences, allowing any manufacturer to build compatible hardware without consortium permission. Third, the license architecture itself functions as a credible commitment: MIT maximizes adoption, while GPLv3’s anti-tivoization clause prevents the vendor from later closing the platform, and the combination strengthens the hostage without foreclosing commercial viability. Fourth, the business model relocates revenue capture to tiered technical support—a competitive, rather than monopolistic, relationship—consistent with West and Gallagher’s (2006) “selling complements” strategy.

We also examine the framework’s alignment with the EU Cyber Resilience Act (Regulation (EU) 2024/2847), showing how the zero-knowledge broker satisfies data-minimisation and attack-surface requirements, and how the self-host option addresses continuity requirements. The paper corrects a prior classification claim: smart door locks fall under Class I of the Regulation’s “important products” tier, not Class II; open licensing reduces the cost of conformity assessment but does not exempt the product from the regulatory gate itself. Generality is bounded by comparison to mining and medical-device sectors, where admission sovereignty is foreclosed by procurement politics and accreditation regimes—suggesting the framework travels most readily to high-consequence but not admission-captured categories.

The paper contributes conceptually by extending hold-up theory from interfirm to firm-consumer relationships and by inverting Williamson’s hostage logic: here, the structurally stronger party (the vendor) posts hostages to reassure the weaker party (the consumer). Empirically, it provides a detailed, verifiable case with an implementation-status register distinguishing specified, built, and field-verified elements. Practically, it offers hardware entrepreneurs and policymakers a vocabulary for distinguishing governance models beyond the binary “open vs. closed.”

Keywords: hold-up problem; transaction cost economics; platform governance; open-source

business models; Internet of Things; EU Cyber Resilience Act; credible commitments

Contents

Abstract	i
1 Introduction	1
2 Literature Review	2
2.1 The Hold-Up Problem and Transaction Cost Economics	2
2.2 Hold-Up in Connected and IoT Hardware	2
2.3 Platform Governance and Admission Control	2
2.4 Open-Source and Commercial-Open-Source Business Models	3
3 Conceptual Framework	3
3.1 Two Axes of Platform Sovereignty	3
3.2 Positioning Existing Governance Models	3
3.3 Hold-Up Avoidance as Unifying Logic	4
3.4 Propositions	4
4 Methodology	5
5 Case Description	6
5.1 Runtime Sovereignty	7
5.2 Admission Sovereignty	7
5.3 License Architecture as Varying-Strength Hostage	7
5.4 Business Model	8
5.5 Status	9
6 Analysis: Business Model as Hold-Up Mitigation	11
6.1 Runtime Sovereignty as Ex-Post Hold-Up Mitigation	11
6.2 Admission Sovereignty as Ex-Ante Hold-Up Mitigation	12
6.3 Open-Source-as-Service as Credible Commitment Device	13
6.4 Comparative Evidence: Documented Hold-Up Failures	14
6.5 A Further Risk Beyond the Documented Cases: Geopolitical Fragility of Shared Platforms	15
7 Discussion	16
7.1 Policy Implications: EU Cyber Resilience Act Alignment	17
8 Limitations and Future Research	21
9 Conclusion	23
References	25
Appendix A: Implementation-Status Register	27

List of Tables

1	Governance Positions on the Two Axes of Sovereignty	3
2	Support Tier Structure	8
3	Revenue Trade-offs: Conventional versus Sovereign Edge Model	9
4	Documented Hold-Up Failures Mapped to the Framework	14
5	Implementation-Status Register (as of July 2026)	28

List of Figures

1	Two-Axis Sovereignty Framework positioning existing governance models. The vacant open-open quadrant is the target of this case.	4
2	Sovereign Edge Runtime Architecture—data sovereignty by design. The root private key is generated on the owner’s device and never transmitted. The broker is untrusted, stores only pairing information, and is structurally unable to read the end-to-end encrypted payload (AES-256-GCM with ECDH-derived keys). The owner can point the bridge to a self-hosted broker, exiting without vendor cooperation. The current state includes a plaintext credential fallback to be removed; the target state is a zero-knowledge broker. Per-device identity prevents clone enrolment while preserving the owner’s reflash right on the consumer tier. The bottom bar summarises security properties: what the vendor <i>cannot</i> do.	10

1. Introduction

Consumer hardware increasingly functions as a subscription relationship wearing the appearance of a one-time purchase. A smart lock or hub depends on an ongoing, vendor-controlled cloud relationship that can be altered unilaterally after sale—the textbook conditions for what transaction cost economics calls the hold-up problem (Klein et al., 1978; Williamson, 1979): once a customer’s investment in installation and habituation is sunk, the vendor holding the complementary asset can appropriate value from it. This is documented, not hypothetical; Section 6.4 examines three cases in detail.

A smart lock, unlike a smart speaker, mediates physical access; an unexpected loss of function is not merely inconvenient but an access or safety failure. This is precisely the category in which the cost of getting governance wrong is highest, and in which a credible, structural defence against hold-up has monetisable value. The smart-lock market occupies a deliberately chosen middle position—what we term a “Goldilocks” case: high-stakes enough for a structural commitment device to matter commercially, but not so thoroughly captured by procurement politics or incumbent accreditation regimes that the open-open quadrant is foreclosed before the architecture can be evaluated on its merits (see Section 7 for boundary conditions). This combination makes it an ideal test case for whether the framework’s mechanisms can function as the theory predicts, without the confounding factors present in either trivial low-consequence categories or fully captured industrial sectors such as mining and medical devices.

The smart-home industry’s two dominant governance responses each solve half the problem. White-label ecosystems built on suppliers such as Tuya impose no barrier to market entry but route device traffic through vendor-controlled cloud infrastructure the customer cannot inspect: open admission, closed runtime—a dependency whose geopolitical dimension Section 6.5 examines directly. Matter, the interoperability standard backed by Apple, Google, Amazon, and the Connectivity Standards Alliance (CSA) they govern, takes the opposite position. It is a genuine engineering achievement: devices are commissioned and controlled locally, with no mandatory cloud round-trip. But market entry is gated behind CSA membership and certification fees priced in the low five figures annually per manufacturer (CSA, n.d.), governed by the same handful of incumbent firms whose own products the standard exists partly to interconnect. Matter does not remove the gatekeeper; it relocates the gatekeeper from the runtime layer to the market-entry layer.

This paper develops a two-axis framework—runtime sovereignty and admission sovereignty—and asks whether a business model can occupy both open positions simultaneously: full local control with no cloud dependency, and no consortium gatekeeping. If such a position is achievable and commercially sustainable, it represents a genuinely new trend in which a device can operate locally and securely without requiring its owner to trust, or a manufacturer to pay, a gatekeeper. The security guarantee this paper is concerned with is specifically a data-sovereignty one: whether the owner’s data remains in the owner’s own hands, at their own disposal, under their own control, rather than passing through and being held by a party the owner cannot verify or exit. That is a narrower and more precise claim than “as secure as,” and it needs to be earned rather than asserted.

The contribution is threefold: conceptually, connecting platform-openness research (Boudreau, 2010) to hold-up theory at the level of an individual consumer’s device; empirically, tracing how the case’s design choices operationalise the framework and situating three documented failures as evidence; practically, giving hardware entrepreneurs and policy audiences a more granular vocabulary than “open.” Version 4.2 of this working paper added verifiable artifacts (Figures 1–2, Appendix A) to move claims from conceptual to checkable; version 4.3 sharpened the regulatory argument in Section 7.1 in the same spirit; version 4.6 added a geopolitical risk analysis of shared white-label cloud platforms (Section 6.5), an explicit chip-level supply-chain disclosure and a

detection-as-complement-to-prevention argument for runtime sovereignty (Sections 6.1 and 8), and a comparative pointer to secure messaging as a second domain in which the open-open quadrant is already realised (Section 7); version 4.7 examines how AI-assisted development shifts the cost calculus underlying Propositions 2 and 3 themselves, strengthening the fork-threat hostage while symmetrically tightening the margin constraint the business model already accepts (Sections 6.3 and 7).

2. Literature Review

2.1 The Hold-Up Problem and Transaction Cost Economics

Transaction cost economics treats firms and markets as alternative governance structures for organising exchange, with the choice driven by the cost of preventing opportunism. Klein et al. (1978) formalised hold-up: when a party invests in a durable, relationship-specific asset worth substantially less in any alternative use, that investment creates an appropriable quasi-rent that a counterparty controlling a complementary asset can attempt to capture once the investment is sunk. Williamson (1979) generalised this into governance theory. Williamson (1983) introduced credible commitments: because contracts are incomplete, parties post hostages—costly, hard-to-reverse commitments forfeited if the posting party behaves opportunistically. A hostage is credible because it is structural, not a promise. The theory was developed for interfirm B2B relationships; extending it to firm–consumer relationships, where the exposed party has none of the contracting options available to a firm, is this paper’s central theoretical move (Section 7).

2.2 Hold-Up in Connected and IoT Hardware

Right-to-repair research addresses a related symptom. Svensson-Höglund et al. (2021) catalogued legal and market barriers restricting a consumer’s ability to extend a device’s useful life. This literature targets physical durability; it says little about the runtime dimension, where a physically intact device depends on a vendor-controlled cloud service that can be withdrawn unilaterally. Local-first software (Kleppmann et al., 2019) supplies architectural vocabulary for that runtime dimension, proposing design principles to restore data ownership and offline operability. That work addressed productivity software, where centralisation failure is an inconvenience; this paper extends the logic to access-controlling hardware, where failure is a safety issue.

2.3 Platform Governance and Admission Control

Boudreau (2010) distinguished granting access (opening markets for outside-built complements) from devolving control (relinquishing authority over platform design). Standards consortia extend this into a monetised, governed register: the CSA’s tiered membership prices market entry for Matter at roughly USD 7,000 per year plus several thousand per product in certification and lab testing (CSA, n.d.), with governing influence held by large incumbents. The gap this paper identifies is the unit of analysis: Boudreau characterises openness from the platform owner’s perspective on complementors, not from a downstream hardware vendor choosing which upstream standard to build into, nor connected to hold-up outcomes for the vendor’s own end customers. Section 3 reframes this as admission sovereignty.

2.4 Open-Source and Commercial-Open-Source Business Models

If a firm forgoes both admission gatekeeping and runtime data lock-in, it forfeits the two most common monetisation mechanisms, raising the question of what remains to sell. West and Gallagher (2006) identified four strategies: pooled R&D, spinouts, selling complements, and attracting donated complements. Selling complements—publishing core technology openly while capturing revenue from an adjacent, harder-to-commoditise asset such as support—is most relevant here. This paper connects that strategy to Williamson (1983) directly: publishing source under an open licence, and structurally forgoing the ability to hold customer credentials hostage, function as literal hostages, verifiable by any technically capable observer. Where classical literature treats the weaker party as posting a hostage to reassure the stronger counterparty, a vendor adopting this model inverts the direction: the structurally stronger party posts a hostage to reassure the weaker consumer.

3. Conceptual Framework

3.1 Two Axes of Platform Sovereignty

Runtime sovereignty concerns who sees, stores, and controls data and functionality while a device is in ordinary use: does routine operation require reaching the vendor’s servers, can the vendor read what passes through infrastructure it operates, and can the vendor unilaterally alter or withdraw access already sold? Admission sovereignty concerns who decides whether a device—and by extension its manufacturer—may exist and interoperate within an ecosystem at all: is there a membership body or certification gate, and is it administered by a market mechanism or by a consortium in which incumbent competitors hold governing votes? The two axes are logically independent: a platform’s position on one predicts nothing about the other, and the practical consequence of each is a specific, nameable form of exposure to hold-up.

3.2 Positioning Existing Governance Models

Table 1 positions existing governance models on both axes. White-label ecosystems (Tuya-style) are open-admission/closed-runtime; interoperability standards (Matter) are closed-admission/open-runtime; vertically integrated proprietary platforms close both. The fourth quadrant—open on both axes—is largely vacant, which is itself informative. Opening both removes mechanisms (admission rents, runtime data control) that would ordinarily fund the coordination costs of maintaining an open standard; Boudreau (2010) finds that granting access accelerates device development but flags coordination costs. How a single small firm can occupy the doubly open quadrant without external subsidy or platform-scale dominance is the puzzle Sections 5–6 address.

Table 1: Governance Positions on the Two Axes of Sovereignty

Governance Model	Admission Sovereignty	Runtime Sovereignty
White-label ecosystems (Tuya-style)	Open	Closed
Interoperability standards (Matter)	Closed	Open
Vertically integrated proprietary platforms	Closed	Closed
Open/Open — this case (“Sovereign Edge”)	Open	Open

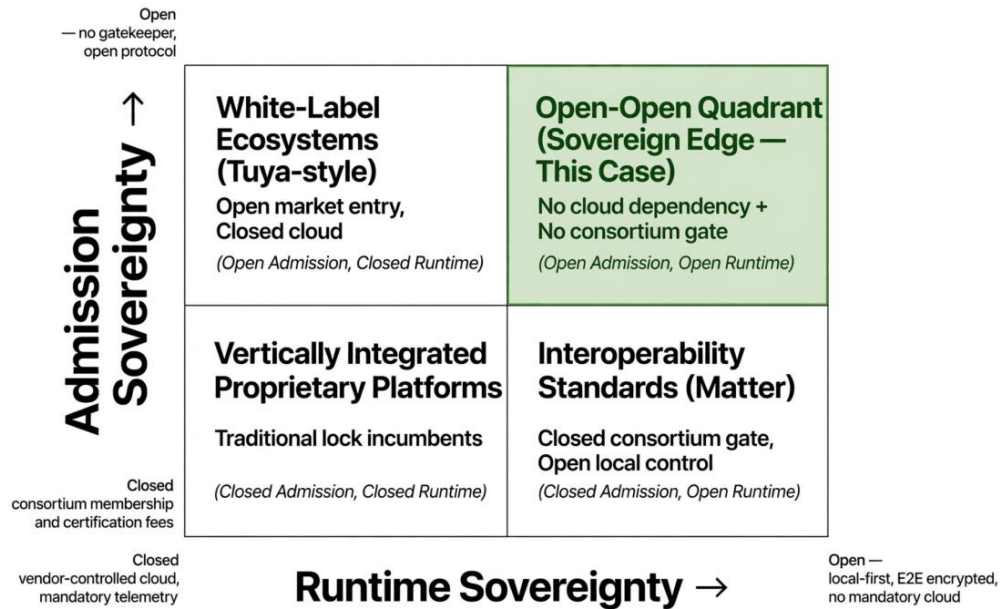
Figure 1: Two-Axis Sovereignty Framework

Figure 1: Two-Axis Sovereignty Framework positioning existing governance models. The vacant open-open quadrant is the target of this case.

3.3 Hold-Up Avoidance as Unifying Logic

Each axis addresses hold-up at a different moment. Admission sovereignty removes ex-ante risk: no proprietary standard narrows the customer's outside options before a dispute, and no certification gate stands in the way of a competing manufacturer offering interoperable hardware. Runtime sovereignty removes ex-post risk: no centralised asset gives the vendor leverage to alter terms after sale, because it does not possess the withheld asset opportunism requires. Neither axis is self-enforcing merely because a firm states an intention to keep it open—what converts stated policy into structural defence is the presence of a genuine hostage in Williamson's (1983) sense: a mechanism costly and independently verifiable to renege on. Published protocol and open-licensed source are hostages against admission-side reversal; an owner-generated root key the vendor never holds is a hostage against runtime-side reversal.

3.4 Propositions

P1. Firms combining open admission (no consortium gatekeeping) with open runtime (no mandatory cloud dependency, no vendor-exclusive control of data or credentials) reduce customer switching costs to the point where post-purchase opportunism becomes structurally unprofitable for the vendor, independent of stated policies.

P2. Because open-admission/open-runtime positioning forfeits two conventional monetisation levers—admission-fee rent extraction and runtime data or service lock-in—a commercially viable business model in this quadrant must relocate revenue capture to a third, comparatively non-appropriable asset, consistent with West and Gallagher's (2006) selling-complements strategy.

P3. Credibility of an open-admission/open-runtime commitment is a function of the irreversibility of the specific mechanisms backing it—whether the cryptographic root key is architecturally

incapable of vendor possession, rather than merely governed by policy stating the vendor will not access it—and not a function of how the commitment is marketed.

4. Methodology

This paper employs a single embedded case study (Yin, 2018), with the unit of analysis being not the firm as a whole but several nested sub-units—firmware and protocol design, credential architecture, license strategy, and support-based business model—each mapped against the propositions in Section 3.4 before synthesis. This design suits a research question for which no single outcome metric captures whether a business model avoids hold-up; the claim rests instead on whether mechanisms at each sub-unit are individually coherent, mutually reinforcing, and consistent with the framework’s propositions.

Case Selection

The case was selected purposively as what Yin terms a revelatory case: a venture designed explicitly around avoiding hold-up on both sovereignty axes, with design reasoning documented contemporaneously rather than reconstructed after the fact. This offers a significant advantage over the three comparative failure cases in Section 6.4, which are observable only as post-hoc events.

However, this justification requires more caution than Yin’s original formulation assumes. The phenomenon studied here was not merely accessed but created by the researcher. To address this tension, the paper adopts a design-science research paradigm (Hevner et al., 2004) alongside the case-study method. Under this framing, the venture is understood as an artefact constructed specifically to instantiate and test the two-axis framework’s propositions. The paper’s proper object of study is the design reasoning captured in that construction—not yet its market reception. This reframing carries a specific methodological payoff: most academic studies of platform governance necessarily observe companies that have already survived market entry, introducing a survival bias that this design avoids. An established platform’s architecture reflects a history of adjustments, some of which may have quietly reintroduced closed mechanisms without being visible to an external researcher. Studying the venture pre-launch, with design reasoning and implementation status dated and disclosed contemporaneously, captures original intent before commercial pressure could test or distort it.

Data Sources

Data are drawn from three sources:

1. Internal technical and business documentation produced and revised across three design iterations, including architecture specifications, protocol definitions, source-code repositories (private at time of writing, with licence headers in place), and a component-level implementation-status register distinguishing specified, built-but-unverified, and field-verified elements (Appendix A).
2. Comparative failure cases drawn from regulatory records—FTC closing letters and press releases—and contemporaneous technology-press coverage, cross-checked across independent outlets.
3. Regulatory analysis based on the published text of Regulation (EU) 2024/2847, European Commission guidance, and legal commentary.

Positionality and Mitigation Strategies

The author’s dual role as researcher and company principal is disclosed upfront. Three strategies mitigate motivated reasoning:

1. **Artifact-based analysis.** The analysis relies on the case’s own dated implementation-status disclosures (Appendix A) and publicly checkable design properties (protocol specifications, license choices, architectural diagrams) rather than promotional claims or future projections.
2. **Explicit limitation disclosure.** The paper reports unresolved risks alongside design strengths, including the plaintext credential fallback still in production, the open key-recovery question, and the absence of admission-side comparative cases.
3. **Scoped claims.** Findings are scoped to design intent and mechanism-level consistency with the framework, not to market validation. The paper does not claim the business model will succeed; it claims the design is coherent and the mechanisms are structurally capable of doing the work the theory predicts.

Mapping to Propositions

The analysis proceeds by examining each sub-unit against the corresponding proposition:

- Runtime mechanisms (Section 6.1) → Proposition 1 (ex-post hold-up mitigation)
- Admission mechanisms (Section 6.2) → Proposition 1 (ex-ante hold-up mitigation)
- License architecture (Sections 5.3, 6.3) → Proposition 3 (credibility as irreversibility)
- Business model (Sections 5.4, 6.3) → Proposition 2 (revenue relocation)
- Comparative cases (Section 6.4) → illustrative evidence of the failure modes the framework predicts

This structure ensures each proposition is tested against specific, observable design choices rather than asserted broadly.

Generalisability Boundary

Single-case designs limit generalisability. This paper does not claim the smart-lock findings apply universally; it instead develops boundary conditions inductively (Section 7), using mining and medical-device sectors as examples of extreme closed-admission ecosystems where the open-open quadrant is commercially unreachable. The framework’s near-term generalisation candidates are narrowed accordingly: categories that resemble smart locks in being high-consequence but not admission-captured—connected consumer medical devices sold direct-to-consumer, industrial equipment in competitive segments—are plausible next cases for comparative study.

5. Case Description

The case is a connected smart-lock platform under development by an Australian hardware and systems-integration firm, targeting residential, hospitality, and—via a distinct hardware tier—defence and government procurement. At the time of writing the product has not shipped and regulatory certification has not yet been obtained; the description below is of the designed system, not a field-validated one.

5.1 Runtime Sovereignty

The bridge device relays commands between the lock and a message broker over an end-to-end encrypted payload; the broker stores only the pairing relationship, never a transaction log, and is designed to be unable to read command content. Owners may point the bridge at a broker of their own choosing, with no firmware change required, which addresses vendor continuity risk: an owner who anticipates vendor failure retains a technical exit requiring no cooperation from the vendor to exercise. Core local control—unlocking at the door and control across the local network—does not require reaching the vendor’s servers. Secondary-user credentials are tokens signed by the owner’s locally generated root key and verified offline against the public key the lock already holds, rather than checked against a server on every use; the private key is generated on, and never transmitted from, the owner’s own device.

5.2 Admission Sovereignty

Server, application, and protocol are published under an open licence (see Section 5.3), with no consortium membership or certification fee required for a customer, integrator, or competing manufacturer to build compatible hardware or software. Supply-chain integrity is addressed through per-device cryptographic identity provisioned at production—a cloned board fails to enrol with any server or pair with any application but is not prevented from booting or being reflashed—rather than through a boot restriction that would also bind legitimate owners. On the consumer and hospitality tier, secure boot is deliberately left unconfigured, which is a genuine security trade-off (Section 6.2) rather than a cost-free choice: the defence and government tier, where secure boot is enabled and firmware vendor-signed, exists for buyers whose threat model weighs physical-tampering resistance above rebuild rights. This split also means the open-admission/open-runtime business model does not carry the venture’s entire revenue case: the closed tier is sold conventionally and functions as a commercial hedge rather than a contradiction.

5.3 License Architecture as Varying-Strength Hostage

Prior versions of this paper described the source as “published under open licence” without specifying which licence. Licence choice directly affects Proposition 3’s irreversibility criterion: different licences create hostages of different strength. This case adopts differentiated licensing:

- **Protocol specification and client SDK: MIT License**—permissive, maximizes admission openness, zero friction for a competing manufacturer to build compatible hardware/software. Weak hostage: the vendor could publish a future version under a proprietary licence, but the community retains perpetual rights to the existing MIT version and can fork.
- **Server, bridge firmware, mobile app: GPLv3**—strong copyleft hostage. GPLv3 requires any distributed modified binary to make corresponding source available, and its anti-tivoization clause (Section 6.2) prevents the vendor from shipping GPLv3 code that only runs if signed by a vendor-only key. This structurally prevents the exact closure scenario the open-innovation literature warns about: a vendor establishing a base and then converting an open platform to closed. If the vendor attempts closure, any customer or competitor can fork the last GPLv3 version and continue service—precisely the credible commitment Williamson describes.
- **Hardware: CERN Open Hardware Licence Version 2—Strongly Reciprocal (CERN-OHL-S v2)**—requires derivative hardware to share schematics under the same licence, preventing closed hardware clones that would undermine supply-chain transparency.

- **Documentation and non-code artifacts: CC-BY-SA 4.0.**

The dual MIT/GPLv3 split is intentional: MIT lowers entry cost (admission sovereignty), while GPLv3 raises exit cost for a vendor attempting reversal (runtime/admission hostage strength). A pure MIT stack would be cheaper to adopt but weaker as a hostage; a pure GPLv3 stack would be the strongest hostage but would deter some commercial integrators. This trade-off is itself evidence for Proposition 2: openness level is a commercial decision with margin implications.

5.4 Business Model

Revenue is designed to come from tiered technical support rather than software or usage data, following West and Gallagher’s (2006) selling-complements strategy. Because the software is openly licensed, a dissatisfied customer can self-host or engage a third party, which keeps the support relationship competitive rather than monopolistic.

Support Tier Structure

The support model is designed around two tiers (Table 2).

Table 2: Support Tier Structure

Tier	Target Customer	Offering	Price Point (Indicative)
Community	Hobbyists, technically proficient owners	Self-hosted broker, DIY installation, community forums, public documentation	Free
Professional	Small businesses, property managers, hospitality operators, residential prosumers	Pre-configured bridge hardware, priority email support, installation guidance, basic SLA, optional cloud-hosted broker (customer-controlled)	AUD 49–99/month per site

The Community tier serves as both a goodwill gesture and a recruitment pipeline. Users who self-host successfully are likely to recommend the product to less technical buyers and may eventually convert to paid tiers as their needs grow. It also functions as a credibility mechanism: the existence of a free, fully functional self-hosted option demonstrates that the vendor is not structurally dependent on locking users into paid services.

The Professional tier captures the majority of the venture’s projected revenue. It targets customers for whom lock failure has operational rather than merely convenience consequences—property managers with hundreds of units, short-term rental operators, and hospitality venues. These customers are willing to pay for pre-configured hardware, guaranteed support response times, and the convenience of a managed broker, while retaining the option to exit to self-hosting if support quality or pricing becomes unsatisfactory.

Competitive Support Market Dynamics

The open license architecture fundamentally changes the support relationship. A customer dissatisfied with the venture’s support pricing has a credible outside option: hire a third-party integrator to provide support, or self-host using the published reference implementation. This constrains the

venture’s pricing power in a way conventional proprietary vendors avoid.

Where a proprietary vendor can raise maintenance fees after the customer has sunk installation costs—classic hold-up—the venture’s pricing is capped by the cost of the next-best alternative. The MIT-licensed protocol specification and SDK mean any systems integrator with sufficient technical capability can offer support services; the GPLv3-licensed server and bridge source mean they can operate those services legally. The venture’s competitive advantage in support is therefore expertise, not exclusivity—accumulated system knowledge, supplier relationships, and ecosystem integration. This is a thinner moat than a proprietary vendor enjoys, which is precisely the point: the structural inability to capture support rents through lock-in is the hostage that reassures customers.

Revenue Trade-offs

The support model’s margin profile differs materially from a conventional hardware-plus-subscription model (Table 3).

Table 3: Revenue Trade-offs: Conventional versus Sovereign Edge Model

Metric	Conventional Model	Sovereign Edge Model
Hardware margin	Thin (competition on bill of materials)	Comparable
Software/subscription margin	High (locked-in base)	Not applicable (no mandatory subscription)
Support margin	Moderate (some lock-in)	Competitive (constrained by outside options)
Customer lifetime value	High (difficult to churn)	Lower (easy to churn)
Vendor switching cost for customer	High (customer bears it)	Low (customer can exit)
Vendor must maintain quality to retain	Moderately	Strongly

The trade-off is explicit: lower margin per customer, offset by lower customer acquisition cost (no need to overcome trust barriers) and potentially higher customer lifetime volume (buyers who would otherwise avoid connected hardware due to lock-in fears).

This is not a claim that support revenue will match what a proprietary vendor could extract from a locked-in base. It is a claim that the credibility benefits—reduced acquisition cost, positive community effects, lower churn—may offset the lower margin per customer. Whether this trade-off is commercially viable at scale is an open empirical question (Section 8), but it is a coherent business strategy grounded in the hostage logic developed in Section 3.

5.5 Status

The cryptographic key-exchange ceremony establishing payload encryption, and the corresponding removal of the currently used plaintext credential fallback on the server, are specified but not yet implemented; local commissioning, mesh networking, and command relay are built and bench-verified. Key recovery for an owner who loses the device holding the root credential, and secondary-user key provisioning at scale, remain open design questions (Section 8). See Appendix A for the full register.

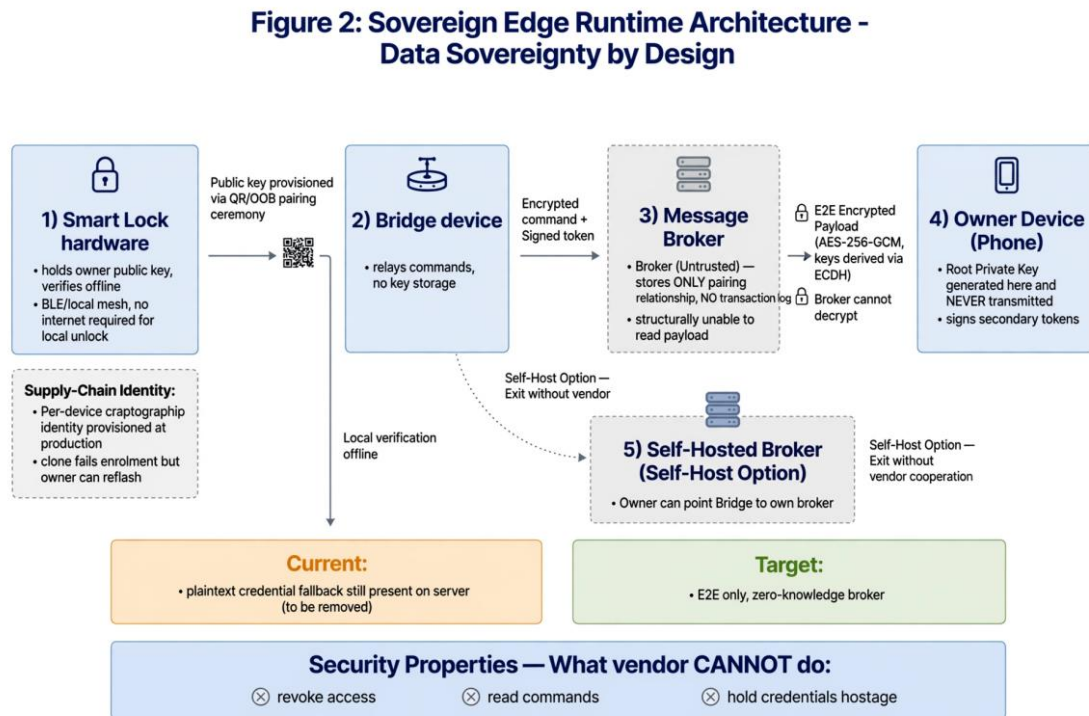


Figure 2: Sovereign Edge Runtime Architecture—data sovereignty by design. The root private key is generated on the owner’s device and never transmitted. The broker is untrusted, stores only pairing information, and is structurally unable to read the end-to-end encrypted payload (AES-256-GCM with ECDH-derived keys). The owner can point the bridge to a self-hosted broker, exiting without vendor cooperation. The current state includes a plaintext credential fallback to be removed; the target state is a zero-knowledge broker. Per-device identity prevents clone enrolment while preserving the owner’s reflash right on the consumer tier. The bottom bar summarises security properties: what the vendor *cannot* do.

6. Analysis: Business Model as Hold-Up Mitigation

6.1 Runtime Sovereignty as Ex-Post Hold-Up Mitigation

The case's runtime mechanisms map directly onto ex-post hold-up risk. Conventional connected-lock architecture centralises two things a vendor could later use opportunistically: data describing when and how a device is used, and credentials determining who may operate it. The case's design removes both from exclusive vendor possession. The broker's designed inability to read command content, and its restriction to storing only pairing rather than a transaction log, mean even continued vendor operation does not grant the vendor a rich behavioural dataset that could later be monetised. More consequentially, the owner-generated root key means the vendor is structurally unable to perform the most direct opportunism: revoking or holding hostage the customer's own access to property. This is the local-first principle (Kleppmann et al., 2019) applied where absence is a security and safety failure. The self-hosting option extends this logic to address vendor continuity risk: the capability whose absence characterises the Nest/Revolv case (Section 6.4).

Runtime sovereignty's ex-post value is not limited to removing the vendor's leverage; it also improves the customer's capacity to detect misbehaviour that no architectural hostage can fully rule out. This distinction matters because none of the defences surveyed in this paper—the owner-generated root key, the zero-knowledge broker, the license architecture—operates below the application layer. None of them can rule out a compromise that exists in the silicon itself: an undocumented low-level command set, a firmware capability the chip vendor never disclosed, or a hardware implant introduced somewhere between fabrication and installation. This is not a risk reserved for one country's suppliers. Leaked NSA documents published in 2014 showed the agency's Tailored Access Operations unit routinely intercepting network equipment—including US-made Cisco hardware—being exported to foreign customers, implanting surveillance tools, and resealing the devices before onward shipment, with Cisco stating it had no knowledge of the practice (Greenwald, 2014). Read against the chip-level dependency disclosed in Section 8, the lesson is that supply-chain compromise is a nation-state capability rather than a property of any single vendor's country of origin, and no amount of application-layer cryptography defends against it once the compromise sits beneath the application.

What runtime sovereignty as designed here does provide is a structurally better position from which to detect such a compromise, even though it cannot prevent one. A conventional cloud-tethered device is expected to communicate constantly with vendor infrastructure, which gives a covert implant's traffic a crowd to hide in. This case's architecture inverts that: because core operation requires no cloud round-trip and the bridge is expected to reach only its configured broker (Section 5.1), the device's legitimate traffic footprint is small and known in advance, which makes any communication outside that footprint—an unexpected outbound connection, an unrequested radio transmission—anomalous by construction rather than needing to be distinguished from a large volume of expected background traffic. The same openness that satisfies Proposition 1 on the admission side also supports detection on the runtime side: because the protocol and reference implementation are public, any sufficiently capable owner or independent researcher can instrument their own device and compare its behaviour against the published specification, distributing the capacity to notice a deviation across every installation rather than concentrating it in the vendor alone—a capability a closed white-label platform does not make available to anyone outside the vendor.

This should be read as a bound, not a guarantee. A sufficiently patient and targeted implant can remain dormant for long periods and beacon rarely; the same leaked material describes exactly this

pattern, with one implanted device reportedly calling back only after several months (Greenwald, 2014). Detection narrows the field of adversaries capable of operating undetected and raises the cost of doing so; it does not eliminate the underlying risk that no application-layer defence, including the ones this paper has otherwise argued for, can close on its own.

6.2 Admission Sovereignty as Ex-Ante Hold-Up Mitigation

Absence of consortium gatekeeping addresses the ex-ante dimension in two ways. First, the customer is not locked into a proprietary standard whose only alternative suppliers are other consortium members subject to the same governance body—the situation Matter’s certification produces, and one Boudreau (2010) predicts will systematically favour incumbent Promoter-tier members. Second, because the protocol and firmware are published (MIT/GPLv3 per Section 5.3), a competing manufacturer can build compatible replacement hardware without the original vendor’s cooperation, which is the outside option determining whether the vendor’s ex-post bargaining position is genuinely constrained.

Declining consortium admission carries an obvious objection: consumers value ecosystem interoperability with Apple, Google, and Amazon, and forgoing Matter could appear to isolate the device. In this case the objection is answered architecturally. Home Assistant’s HomeKit Bridge integration exposes local entities to the Apple Home app over the HomeKit Accessory Protocol (HAP) under the protocol’s non-commercial track—the same mechanism Homebridge uses—which requires no manufacturer certification because the vendor never implements HAP directly; Home Assistant, a separate non-commercial project the owner chooses to run, is the party making the claim on the protocol’s non-commercial terms. The dependency is additive rather than structural: if the integration changes or Apple narrows the non-commercial track, the lock continues to operate exactly as before, because no part of core function passes through that path, in contrast to the Matter architecture, where the same change would sit directly in the commissioning path. Equivalent routes exist to Google and Amazon with more configuration. This is not equivalent to out-of-box certified interoperability—it requires the customer to run their own hub, and the vendor cannot use certification branding—but it demonstrates that admission sovereignty and ecosystem reach are not the same trade-off Table 1 might suggest.

The steelman case for admission gatekeeping deserves a direct response: device attestation exists to prevent counterfeit hardware from silently joining a network. The case’s response separates two problems. Supply-chain integrity—preventing unauthorised surplus production—is addressed through per-device identity provisioning, a mechanism aimed at the manufacturer relationship, not ecosystem admission; a cloned board fails to enrol but is not prevented from booting or being reflashed. This echoes Klein et al.’s (1978) observation that the appropriate response to opportunism risk should be matched to the specific relationship in which the risk arises. Leaving secure boot unconfigured on the consumer tier is itself a security trade-off: an unlocked boot is open to a motivated adversary with physical access—firmware could be replaced with a compromised build persisting across reboots while appearing normal. The framework’s response is the same as applied to the defence tier—a threat-model choice rather than a universal answer—but the choice has a real cost: a buyer for whom physical-tampering resistance matters more than rebuild rights is a buyer this configuration is not designed for, and the defence-tier hardware, with secure boot enabled, exists precisely for that buyer.

Because the anti-clone mechanism operates through cryptographic identity rather than boot restriction, its security guarantee is independently verifiable by a third party inspecting the published protocol (MIT-licensed spec), whereas the boot-restriction claim consortium certification relies on is not verifiable without trusting the certifying body’s testing. The admission-sovereignty

mechanism is therefore not simply cheaper than consortium certification but differently verifiable, which is the property Section 3.3 identifies as the actual determinant of a credible commitment's strength. GPLv3's anti-tivoization clause further strengthens this: the vendor cannot legally ship GPLv3 bridge firmware locked to vendor-only signing keys while withholding the ability to run a modified version.

6.3 Open-Source-as-Service as Credible Commitment Device

West and Gallagher's (2006) selling-complements strategy functions simultaneously as a Williamson-style (1983) hostage mechanism. Three design elements can be read this way. Publishing server and firmware source under GPLv3 is a hostage against admission-side reversal: a vendor quietly restricting compatibility would be doing so visibly against a public reference. The owner-generated root key is a hostage against runtime-side reversal. The absence of a secure-boot restriction on the consumer tier is a hostage against the vendor's own future temptation to convert an open platform into a closed one after establishing a customer base—a temptation the open-innovation literature identifies as a standing risk, and one this design forecloses structurally rather than through a policy promise not to do so.

This reading clarifies the inversion flagged in Section 2.4. In Williamson's (1983) original formulation, hostages are typically posted by the party with less structural power to reassure the stronger counterparty. Here, the vendor—the party who designs the product, sets the price, and controls the initial terms—posts hostages to reassure the comparatively weaker party: an individual consumer who, in a conventional connected-hardware relationship, has little recourse once the purchase is complete. Section 7 returns to this inversion as part of the theoretical contribution.

The revenue implication, consistent with Proposition 2, is that the design forfeits both admission-fee and runtime-data-lock-in revenue and relies instead on a competitive support market. This is a genuine commercial trade-off: a customer dissatisfied with support pricing can self-host or hire a third party, which caps achievable support margin at whatever the competitive market bears, rather than what a locked-in base would tolerate.

Proposition 3 ties a hostage's credibility to the cost of exercising it, and that cost is not fixed across time. When Williamson (1983) developed the concept, and even when West and Gallagher (2006) analysed selling complements as a strategy, forking and independently maintaining a full embedded-firmware-plus-cloud-plus-mobile stack required a team large enough that the threat, while structurally real, was expensive enough to exercise that a vendor could reasonably discount it. AI-assisted software development changes this specific calculus, not the underlying theory. A GPLv3 fork of the server and bridge source, or an MIT-licensed reimplement from the published protocol, is now plausibly within reach of a single capable developer working with AI coding tools across the parts of the stack that are conventional application logic—API surfaces, mobile UI, deployment tooling—rather than requiring the multi-person team that made the threat mostly theoretical in the period the underlying theory was developed. This does not extend uniformly across the stack: the cryptographic protocol correctness, key-management design, and real-time firmware constraints this paper's own status register (Appendix A) shows taking longest to move from specified to field-verified are precisely the parts of the system where AI assistance offers the least acceleration, because their difficulty lies in getting a small amount of logic exactly right under adversarial conditions rather than in writing a large volume of conventional code. The practical effect is therefore to lower the cost of the fork threat unevenly, concentrated on exactly the boilerplate-heavy portions of the stack a competitor would need to replicate quickly, while leaving the harder security-critical core roughly as costly to get right as before. Even a partial reduction in fork cost strengthens the credibility Section 3.3 and Proposition 3 describe: a hostage whose

exercise cost falls is, by the paper’s own logic, a stronger hostage, independent of anything the vendor does or says.

6.4 Comparative Evidence: Documented Hold-Up Failures

Three documented cases illustrate the mechanism the framework is designed to prevent.

Nest/Revolv. Google’s Nest acquired smart-home hub maker Revolv in 2014; in February 2016 it announced that, as of 15 May 2016, previously sold Revolv hubs—marketed with a “free lifetime service subscription”—would cease to function (Federal Trade Commission [FTC], 2016). The FTC opened an investigation, concluding that staff were concerned reasonable consumers would not have expected the hubs to become unusable, and that unilaterally rendering them inoperable risked causing substantial, unavoidable consumer injury; the matter was closed after Nest offered full refunds (FTC, 2016). In terms of Section 3.3, this is a clean instance of runtime sovereignty’s complete absence: entire value contingent on a single vendor-controlled server whose continued operation was promised only as a policy commitment—“lifetime subscription”—with no architectural hostage backing the promise.

LockState/RemoteLock. In August 2017, an over-the-air firmware update pushed by LockState to the RemoteLock 6i model—sold predominantly to Airbnb hosts—contained a defect that permanently disabled the smart-code access function on at least 500 units, with roughly 200 host businesses directly affected; recovery required units to be physically returned for reflashing, taking five to seven days, because the defect could not be corrected remotely once the affected units lost server connectivity (Cimpanu, 2017). Physical keys continued to work throughout. This is a more nuanced instance: a runtime fallback existed at the mechanical layer, while the smart functionality that constituted the actual value proposition remained entirely hostage to the vendor’s server infrastructure and firmware-push pipeline, with no local alternative to restore functionality independently.

Ring/Amazon. In May 2023, the FTC announced a settlement following findings that Ring employees and third-party contractors had, for a period, unrestricted technical ability to access and download customer video—including footage from cameras installed in bedrooms and bathrooms—and that video was additionally used to train algorithms without consent; the settlement required Ring to pay USD 5.8 million in refunds (FTC, 2023). This illustrates the distinction between policy-level and architecture-level runtime commitment sharply: the failure was not the absence of a stated privacy commitment, but the presence of a technical capability—employee access to unencrypted video—that made the outcome possible regardless of policy. A promise not to view footage is not a hostage if the vendor retains the technical capacity to view it; only an architecture in which the vendor never holds the decryption key functions as one.

Table 4: Documented Hold-Up Failures Mapped to the Framework

Case	Axis	Mechanism Absent
Nest/Revolv	Runtime	Policy-level lifetime promise; no architectural hostage or self-hosting fallback
LockState/RemoteLock	Runtime (partial)	Mechanical fallback existed; smart functionality remained server-dependent
Ring/Amazon	Runtime	Policy-level privacy commitment undermined by retained technical access capability

Read together, the three cases share a structure the framework predicts: in each, the vendor’s stated intentions at the time of sale were not obviously dishonest, and harm materialised later, once

the customer's investment was sunk and the vendor's retained technical capability made a different course available, or made an unintended failure unrecoverable without vendor cooperation. None involved the runtime-sovereignty mechanism described in Section 6.1 that would have given affected customers an independent path back to functionality.

Two scope limits are worth naming. First, all three cases are runtime-sovereignty failures; none illustrates an admission-sovereignty failure in which consortium gatekeeping itself produced harm. The paper is not aware of a comparably documented case fitting that description, which may reflect that admission-side harms are less visible to press and regulators than an abrupt loss of function, rather than that they do not occur; the comparative evidence should be read as evidence for the runtime axis specifically, not for admission-side claims, which rest on the cost and governance structure described in Section 2.3 rather than a documented failure case. Second, Ring's violation is a privacy and access-control breach rather than hold-up in the stricter sense—no price was renegotiated and no functionality was withdrawn—and is included specifically for what it demonstrates about the gap between policy-level and architecture-level commitment, not as a third instance of the same opportunism pattern.

6.5 A Further Risk Beyond the Documented Cases: Geopolitical Fragility of Shared Platforms

The three cases in Section 6.4 share a structure worth naming explicitly: each is a single vendor's own commercial decision, affecting that vendor's own customer base, and each has already occurred. A further risk is worth flagging precisely because it does not share that structure. It has not, at the time of writing, occurred, and it would not be a documented case in the sense Section 6.4 uses the term—but it is not merely speculative, either, and its potential scale is large enough that omitting it would understate what the open-admission/closed-runtime quadrant in Table 1 actually exposes a customer to.

Tuya and TTLock, two of the dominant white-label cloud platforms underlying a large share of the global smart-lock and smart-home hardware market, are both PRC-domiciled operators. Tuya additionally trades as a listed company (NYSE: TUYA; HKEX: 2391) through a variable-interest-entity structure controlling a wholly owned PRC subsidiary, and its own annual report identifies continuing US–China friction and tariff measures, which the company states have disrupted its international business since April 2025, as a principal external risk to its operations (Tuya Inc., 2026). This is not a concern invented for this paper: cybersecurity researchers and US policymakers have already raised, in the specific context of Tuya, whether continued dependence on a PRC-governed platform controlling a very large number of connected devices in Western homes is itself a security exposure, including a 2020 legislative proposal aimed at restricting exactly this category of high-risk foreign IoT software (VOA News, 2021).

What distinguishes this risk from the three documented cases is its correlation structure rather than its underlying mechanism. Nest/Revolv, LockState/RemoteLock, and Ring/Amazon each stranded the customers of a single brand, because runtime sovereignty in each case was held by one vertically integrated vendor. A white-label platform inverts this relationship between admission and risk. Admission is open—any manufacturer can build a lock on Tuya's or TTLock's platform without that manufacturer itself holding runtime control—but the resulting runtime dependency is pooled across every brand built on the platform. Tuya's own materials describe a footprint spanning hundreds of thousands of registered developers and industry-analyst rankings placing it first in the global smart-home and smart-business IoT platform-as-a-service market by device count (Tuya Inc., n.d.). A sanctions action, export-control restriction, data-localisation mandate, or unilateral platform shutdown affecting a platform of that scale would not strand one vendor's

customers; it would simultaneously strand the customers of every brand built on it, at a magnitude the three single-vendor cases in Section 6.4 do not approach. In the vocabulary of Section 3.1, this is what open admission without open runtime actually buys a customer: freedom for many manufacturers to enter the market, purchased at the cost of concentrating all of their customers' runtime dependency onto the same handful of foreign-controlled servers—the opposite of the diversification open admission might appear to offer at the hardware layer.

This risk also sharpens the continuity argument developed in Section 7.1(2). The CRA's concern that a product should not become unusable if the manufacturer stops providing updates or support is naturally read as addressing vendor insolvency or a change of business strategy, the pattern Nest/Revolv illustrates. A platform-level geopolitical disruption is the same continuity failure at a different scale and with a different, and for an individual manufacturer largely uncontrollable, cause: the manufacturer may be entirely solvent and willing to continue support, and its devices may still lose remote-control functionality because the shared cloud layer beneath the whole white-label ecosystem becomes unreachable or is required by law to be severed. Runtime sovereignty of the kind described in Section 5.1—core local unlock that never depends on any server, and a broker an owner can rehost anywhere, including outside the jurisdiction of any single government—is, among the mechanisms surveyed in this paper, the only one that removes this specific failure mode structurally rather than merely mitigating its consequences after the fact. It is also, on this reading, a further argument for why open admission (Table 1's vertical axis) is not a substitute for open runtime (its horizontal axis): the white-label quadrant satisfies the former while leaving the latter—and with it, exposure of exactly this kind—entirely open.

7. Discussion

Theoretical contribution. Extending hold-up theory from interfirm to firm–consumer relationships requires precision. Classical theory's bar for asset specificity is strict, and a homeowner's smart-lock skills transfer reasonably well to a different vendor's product—a more general asset than Klein et al.'s stamping die. What is genuinely transaction-specific is narrower: installed hardware, credentials, and automations built around one device, which together constitute an appropriable quasi-rent operationalisable as physical replacement cost plus reconfiguration cost. Broader behavioural investment is better read as a switching cost compounding that narrower quasi-rent than as strict asset specificity, and the paper keeps the distinction explicit. The more consequential departure is the direction of hostage-posting (Section 6.3): here the structurally stronger party posts the hostage, because the ordinary consumer has none of the contracting or litigation options available to the hold-up-exposed firm in the original account, which makes architectural hostages—mechanisms that remove capability rather than merely constrain use—comparatively more important in this setting than in the interfirm settings the theory was built to explain. Whether the direction generalises to other consumer markets with severe post-purchase dependency—connected medical devices, financial-account software—is a question for future comparative work.

Managerial and boundary implications. Proposition 2's revenue relocation is a real margin constraint, not a costless stance: a dissatisfied customer's ability to self-host caps achievable support pricing at whatever the competitive market bears. The trade-off is more likely commercially defensible where three conditions hold together: vendor failure has severe consequences, buyers are sophisticated enough to evaluate architectural rather than marketing claims, and the support relationship is ongoing enough to sustain a service business rather than a single transaction. Access-control hardware, sold in part to institutional and professional buyers, plausibly satisfies all

three; commodity, low-consequence IoT categories—decorative lighting, simple sensors—plausibly satisfy none, which bounds the framework’s commercial, though not descriptive, applicability. The license choice in Section 5.3 is part of this commercial calculus: GPLv3 strengthens credibility (a strong hostage) but may deter integrators who fear copyleft obligations; MIT maximizes adoption but weakens the hostage. The differentiated MIT-for-spec/SDK plus GPLv3-for-server/bridge split is an explicit attempt to balance P3’s credibility requirement against P2’s margin constraint.

The same AI-assisted development cost reduction discussed in Section 6.3 as strengthening the fork-threat hostage cuts against the venture in its capacity as a margin constraint, and the two effects should not be read as net positive without qualification. A lower cost of forking the venture’s own reference implementation is, from a different vantage point, a lower cost of building a rival implementation from nothing—open or closed, faithful to this paper’s framework or not. Proposition 2 already predicts that this quadrant’s revenue must come from expertise rather than exclusivity; the same forces that make the GPLv3 hostage more credible also shrink the time during which accumulated system knowledge remains a scarce, monetisable asset relative to a fast-moving imitator, whether that imitator is a customer exercising the fork option this paper treats as desirable, or a new entrant with no relationship to the venture at all. The paper’s existing argument that support-market competition, rather than lock-in, disciplines pricing (Section 5.4) already anticipates thinner margins than a proprietary vendor enjoys; a lower cost of market entry generally tightens that discipline faster than the venture’s original business-model projections assumed, which is a reason for a realistic revenue timeline rather than a reason to abandon the model. Whether the venture’s advantage in this environment comes from being first, or has to come from something more durable—accumulated integration relationships, a support reputation that is itself costly to fake, or continued technical leadership on the security-critical core the previous paragraph argues is harder to replicate quickly—is an open strategic question this paper’s design-stage evidence cannot yet answer.

Broader significance. The governance question is not confined to smart locks, and its resolution matters beyond a single firm’s commercial prospects. The introduction posed a specific claim to be earned: a device can operate locally, encrypted, and always available, the way a governed platform standard’s architecture provides, while additionally keeping the owner’s data in the owner’s own hands—never transmitted to, or held by, a party the owner did not choose and cannot verify—without requiring trust in a vendor’s cloud or payment to a consortium gatekeeper. The analysis in Section 6 supports that claim at the level of design: the case’s runtime mechanisms reproduce the local-first functional guarantees Matter offers, while going further on the data-sovereignty dimension, since the Matter architecture does not itself require that a device’s data never pass through a vendor-controlled service; the case’s admission mechanisms separately remove the certification economics that convert market entry into a five-figure annual cost governed by incumbent competitors. If this position is commercially sustainable—an open empirical question Section 8 identifies—it represents a genuine alternative trend in connected-hardware governance: data sovereignty and owner control delivered as an architectural property rather than a policy promise, at a cost structure not dependent on consortium permission.

7.1 Policy Implications: EU Cyber Resilience Act Alignment

Right-to-repair regulation (Svensson-Höglund et al., 2021) addresses hold-up’s physical-repairability dimension but says little about whether a functioning device’s software-mediated capability can be withdrawn by a vendor with exclusive control of the service it depends on. The EU Cyber Resilience Act (CRA) directly addresses this gap and strengthens this paper’s policy contribution.

The CRA came into force on 10 December 2024 as Regulation (EU) 2024/2847, with main obligations applying from 11 December 2027 after a transition period. It is a horizontal regulation requiring manufacturers to incorporate cybersecurity measures into product design and maintenance, to document what is inside their products, to manage vulnerabilities throughout the product lifecycle, and to provide security updates. For this framework, three CRA provisions are directly relevant, and each is stated below with the correction or sharpening this revision makes explicit.

- (1) **Data minimisation, transparency, and attack-surface reduction.** The Regulation’s essential requirements direct manufacturers to design products that expose no more attack surface than necessary and to limit data collection—including usage and diagnostic data—to what is adequate, relevant, and necessary for the product’s intended purpose. This paper’s runtime architecture speaks to both halves of that requirement, not only the transparency half emphasised in version 4.2. The broker’s designed inability to decrypt command content, and its restriction to storing only the pairing relationship rather than a transaction log, mean the aggregation point a compromised or subpoenaed server would otherwise represent simply does not exist: there is no accumulated behavioural dataset to expose, because none is collected. This is the data-minimisation requirement satisfied at the architecture layer rather than the policy layer—the same distinction Section 6.4’s Ring case turns on. Where Ring’s stated commitment not to view footage was a policy commitment sitting on top of retained technical access, the zero-knowledge broker removes the access capability itself, which is what a data-minimisation requirement grounded in attack-surface reduction, rather than in stated intent, actually asks for. Zero-knowledge broker design with an owner-held root key is evidence of this kind of compliance.
- (2) **Continuity and local-control requirement.** CRA recitals emphasise that products with digital elements must remain operable and securely maintainable even if manufacturer support changes; current industry guidance interpreting these recitals treats “remains operable” as a design standard the product must satisfy, not a promise that vendor infrastructure will remain available indefinitely—a product need not survive vendor withdrawal by luck; it must be designed to. This paper states explicitly, rather than leaving implicit, which mechanisms are doing this work: the self-host option is the direct route through which the case satisfies this requirement, because the bridge can be repointed at any compatible broker—including a self-hosted one—via a configuration value, which means an owner is not dependent on the original vendor’s continued solvency, and neither is a hobbyist or integrator maintaining the device after the vendor exits the market. Local offline verification, where the lock checks signed tokens against a stored public key without reaching any server, supplies the same guarantee for the core unlock function specifically. Read together, these two mechanisms convert “the product should not become unusable if the manufacturer stops providing updates or support” from a recital-level aspiration into a testable design property: does the device continue to perform its core function, verifiably, when the vendor’s servers are switched off entirely? For this case the answer is yes for local unlock and self-hosted broker use; the current gap, noted throughout this paper, is the plaintext credential fallback still present on the server pending the end-to-end implementation recorded in Appendix A. A product that bricks outright when the vendor’s server disappears, as in Nest/Revolv, would fail this continuity standard as currently interpreted.
- (3) **Admission barrier and SME competitiveness (corrected in version 4.3).** Annex III of the Regulation lists smart-home products with security functionalities—explicitly including smart door locks—as “important products with digital elements.” Verification against the Annex

III text and current legal commentary (European Commission, n.d.; Pillsbury Winthrop Shaw Pittman LLP, 2025) places this product category in Class I of that tier rather than the higher-risk Class II tier, which covers products such as firewalls, intrusion-detection systems, hypervisors, and tamper-resistant microprocessors. This distinction matters for the strength of the claim version 4.2 made: Class II products face a mandatory third-party conformity assessment route regardless of the manufacturer's licensing choices, whereas Class I products may in principle use a self-assessment route—CE marking without a notified body—provided the manufacturer fully applies the relevant harmonised standards, common specifications, or an EU cybersecurity certification scheme covering the applicable essential requirements. Where such standards are not yet finalised or not fully applied—a live risk during the Regulation's transition period—a notified body's involvement can still be required even for a Class I product. The earlier framing in this section overstated what open licensing achieves: publishing the protocol and reference implementation under MIT/GPLv3 does not exempt this product from the CRA's conformity-assessment gate, and does not by itself convert a self-assessment obligation into a certainty. What it does do is lower the cost of the underlying compliance work—a public, auditable reference implementation and a software bill of materials reduce the documentation and audit-preparation burden that any conformity route, self-assessed or third-party, requires; a small manufacturer building against a published, already-scrutinised protocol starts from a materially smaller gap than one building a proprietary stack from scratch. The revised claim is narrower than the original: the open license architecture reduces the cost of reaching whichever conformity route applies to this product category; it does not eliminate the gate itself, and the two should not be conflated.

In short, the CRA converts what this paper frames as competitive differentiation (data sovereignty as monetisable value) into an emerging regulatory requirement, subject to the correction above about which conformity route actually applies. The framework's two axes give policymakers vocabulary: right-to-repair covers the physical dimension, the CRA covers runtime security, transparency, and (for this product class) a Class I conformity route, but neither explicitly addresses the admission dimension—who decides which manufacturers may compete. Pairing repairability mandates with data-portability/local-control requirements (runtime) and recognition of open, verifiable implementations as compliance-preparation evidence, rather than as a substitute for conformity assessment itself (admission), would extend the existing logic to both axes this paper makes explicit. Funding and procurement environments that treat consortium-gated interoperability as the only credible path to platform-grade security overlook the architectural alternative described here, even where—as the correction above shows—that alternative still has to clear the same regulatory gate as everyone else.

Generalisability. The framework's commercial boundary conditions are sharper once two distinct failure modes are named, rather than treated as a single undifferentiated caveat. The first, present in earlier versions of this paper, is that stakes can simply be too low: commodity, low-consequence IoT categories—decorative lighting, simple sensors—plausibly lack buyers sophisticated enough to price an architectural trust commitment into a purchase decision, which removes the commercial basis for Proposition 2's revenue relocation even where the governance logic still describes the market accurately. The second, made explicit in this revision, is that a market can be too thoroughly captured on the admission axis for the open-open quadrant to be commercially reachable at all, independent of stakes.

Mining and medical devices are instructive here precisely because they show what full closed-admission capture looks like, and why the open-open quadrant is not merely unattractive in such markets but functionally foreclosed. In both sectors, incumbent suppliers occupy a position

functionally equivalent to the consortium gatekeeper described in Section 3.1, but without a consortium's formal, at-least-nominally-contestable governance structure: qualification as an approved or preferred supplier is itself the market-entry gate, and that gate is guarded through closed-loop sourcing—hospital group-purchasing arrangements and long-standing OEM relationships in medical devices, and safety-case accreditation tied to a specific incumbent's equipment history in mining—that a new entrant cannot satisfy merely by publishing a better protocol. Conservative safety regulation, which exists for good reason in both sectors, is simultaneously the mechanism incumbents rely on to hold that gate: approval pathways calibrated to an incumbent's existing track record function as an artificial barrier to entry that a technically superior but unproven open architecture cannot cross on technical merit alone, however strong its Williamson-hostage properties are on paper. This is regulatory capture in the classical sense, and it means admission sovereignty in these sectors is not primarily a licensing or protocol-openness problem the way it is for smart locks—it is a procurement-and-accreditation-politics problem this paper's framework does not, by itself, solve.

The smart-lock market occupies a materially different position, which is what makes it a useful test case for the framework rather than an idiosyncratic one. It shares the high-stakes property that makes governance failures costly—Section 1's observation that a lock, unlike a smart speaker, mediates physical access and safety rather than mere convenience—but it has not hardened into a closed-loop procurement ecosystem the way mining and medical equipment have. There is no dominant group-purchasing structure, no small set of accredited-only suppliers, and no safety-case regime that ties certification to a specific incumbent's history rather than to a product's demonstrated properties. In the vocabulary developed here, smart locks are high-consequence but not closed-admission-captured, which is precisely the combination the open-open quadrant needs in order to be commercially testable: severe enough consequences that a credible, structural commitment device has monetisable value (this section's managerial discussion), but not so thoroughly captured that admission sovereignty is foreclosed by procurement politics before the architecture argument can be evaluated on its own merits. This is the paper's Goldilocks framing: the category is a deliberate middle case, chosen because it is where the open-open proposition is genuinely testable rather than trivially true (low stakes, any governance model will do) or trivially foreclosed (high stakes captured by an accreditation moat no architecture can cross).

This sharpens, rather than narrows, the earlier generalisability claim. Categories that resemble smart locks in this specific respect—real safety or access consequences, but a market structure not yet locked behind incumbent-calibrated accreditation—are the ones to which the framework should be expected to travel with least modification: connected consumer medical devices sold direct-to-consumer rather than through hospital procurement, and industrial or agricultural equipment sold in structurally competitive rather than safety-case-gated segments, are more plausible near-term candidates than mining plant or hospital-procured medical devices as those categories are currently governed. Extending the framework into fully captured categories is a different and harder project: it would need to explain not just how a firm posts credible admission- and runtime-sovereignty hostages, but how it does so in a market where the accreditation gate itself, rather than the vendor's own architecture, stands between the firm and the customer—a question this paper's single-case, pre-market design is not positioned to answer, and one this paper treats as an explicit scope limit rather than an oversight. This is a boundary condition on commercial viability, not on descriptive power: the two-axis distinction characterises any connected-hardware governance structure regardless of whether occupying the doubly open quadrant happens to be reachable within it, which keeps the framework useful as an evaluative tool for policy and procurement audiences even in categories—mining and medical devices included—where the specific business-model argument developed here would not, on its own, apply.

A closely related domain illustrates the open-open quadrant already realised, rather than merely proposed. Secure messaging offers a near-exact structural analogue: end-to-end encryption is runtime sovereignty applied to communications rather than device control, and open, federated protocols—Matrix’s homeserver federation, or peer-to-peer designs such as Briar that require no central server at all—extend admission sovereignty to who may run infrastructure, not only who may manufacture hardware. The domain also surfaces a variant of admission sovereignty this paper’s manufacturer-centric framing does not fully capture: centralised platforms can deplatform individual speech acts unilaterally, a censorship-flavoured hold-up that federated or self-hostable alternatives resist in the same way this case’s self-host option resists vendor withdrawal (Section 6.1). A full treatment of messaging as a second case is beyond this paper’s scope, but the parallel is worth naming: unlike mining or medical devices, where the open-open quadrant is foreclosed, secure messaging shows it is commercially and technically reachable outside connected hardware entirely.

8. Limitations and Future Research

The analysis rests on a single, pre-revenue, pre-launch case, and this is worth engaging on its own methodological terms rather than only noting it and setting it aside as a weakness. Two readings of pre-market status are available. The first treats it as a straightforward evidentiary gap: claims about a product not yet sold cannot be checked against market outcomes, full stop. The second, adopted explicitly from this revision onward, reads the case through a design-science research paradigm (Hevner et al., 2004), building on the revelatory-case framing already invoked in Section 4: the venture is not an unfinished example of an existing category but an artefact constructed specifically to instantiate and test the two-axis framework’s propositions, and the paper’s proper object of study is the design reasoning captured in that construction, not—yet—its market reception. Read this way, the burden the venture has taken on—building hardware, firmware, server, mobile application, and the beginnings of a self-hosting and integrator community as a single small team, rather than licensing an existing stack and modifying it—is not incidental difficulty to be apologised for; it is the condition under which the case can exist as a revelatory instance of the open-open quadrant at all, since no full-stack open implementation of this kind was available to study by observing an established incumbent instead.

This reframing carries a specific methodological payoff rather than being merely rhetorical. Most academic studies of platform governance necessarily observe companies that have already survived market entry, which introduces a survival bias this paper’s design avoids: an established platform’s publicly observable architecture reflects a history of adjustments, some of which may have quietly reintroduced closed-admission or closed-runtime mechanisms an early design explicitly rejected, without that reversal being visible to a researcher working only from the outside. Studying the venture pre-launch, with design reasoning and implementation status dated and disclosed contemporaneously (Section 4; Appendix A), captures original intent before any such drift could occur and before commercial pressure has had the opportunity to test it—a vantage point largely closed to research conducted on already-established platforms. The trade-off, acknowledged squarely rather than absorbed into the reframing, is that this vantage point cannot yet speak to whether the design survives contact with the market; that limitation is separate from the pre-market-status question and is addressed directly below.

Most mechanisms in Section 5 are specified rather than field-validated, and the paper’s claims are scoped to design intent and consistency with the framework, not to realised outcomes. The author’s positionality as the case firm’s principal (Section 4) means findings about mechanism

design should be weighted more heavily than any claim about eventual market success would be, precisely because the former can be checked against artifacts—published protocol, a dated implementation-status table (Appendix A), independently reported comparative cases—while the latter cannot yet be checked against anything for a product not on the market. Two specific design questions remain genuinely open rather than merely undocumented: key recovery for an owner who loses the root-credential device (current sketch: Shamir Secret Sharing 2-of-3 backup to trusted contacts or an offline QR code, plus a destructive re-pairing ceremony that invalidates the old root public key on the lock via a physical button press—to be specified), and the comparative evidence base for admission sovereignty specifically, which rests on the cost and governance structure (Section 2.3) rather than a documented admission-side failure case of the kind available for the runtime axis (Section 6.4).

A related, chip-level dependency is worth naming explicitly rather than left implicit in the supply-chain discussion of Section 5.2. The venture’s hardware runs on Espressif’s ESP32 family—specifically the ESP32-P4 as the application/security processor and the ESP32-C6 as its wireless companion, connected over SDIO—a choice that trades the option of designing silicon from scratch for the practical reality that no small hardware venture can independently fabricate or fully audit a commodity SoC. This dependency is not merely theoretical: in March 2025, security researchers disclosed 29 undocumented Bluetooth HCI commands in the original ESP32 chip, tracked as CVE-2025-27840, capable of unauthorised memory access and device impersonation (BleepingComputer, 2025). Espressif’s official response characterised the issue as internal debug tooling rather than a deliberate backdoor, committed to a firmware fix removing the commands, and confirmed that the C6, S-series, and H-series chips were not affected (Espressif Systems, 2025). The episode is instructive independent of its resolution: it demonstrates that undocumented low-level functionality in commodity wireless silicon is a live category of risk rather than a hypothetical one, and—per the discussion in Section 6.1—that no amount of application-layer cryptography can defend against a compromise that exists beneath the application layer, in ROM or radio firmware the venture cannot inspect.

The design response is architectural rather than assurance-based. The root private key is generated on, and never transmitted from, the owner’s phone (Section 5.1), so the lock- and bridge-side silicon is never trusted with the one secret whose compromise would be catastrophic, regardless of what a fully compromised chip could otherwise do. The P4/C6 split reinforces this separation of concerns: the wireless stack—the more exposed surface, per the case above—runs on the C6 co-processor, isolated by the SDIO boundary from the P4’s hardware key-management unit and digital-signature peripheral, which are intended to hold and operate on device-identity key material. At the time of writing the venture is evaluating a discrete secure element for per-device identity keys outside either Espressif chip, together with a hardware real-time clock and monotonic counter independent of network time, so that replay-window enforcement does not rest on a time source a physically present adversary could otherwise roll back. None of this proves the absence of an undisclosed hardware vulnerability in any component—that is, in general, unprovable—but it bounds what a worst-case chip-level compromise could achieve, which, combined with the detection posture discussed in Section 6.1, is the only defence available against a category of risk no vendor’s software layer can rule out on its own.

Single-case designs limit generalisability beyond the boundary conditions discussed qualitatively in Section 7. Future work should pursue comparative multi-case analysis as open-admission/open-runtime ventures emerge in other categories, which would allow those boundary conditions to be tested rather than asserted from one instance; longitudinal follow-up once this case’s credential layer ships and the product reaches market, to test whether the framework’s mechanisms survive the transition from specification to field deployment; and direct empirical

testing—survey or experimental—of whether consumers and institutional buyers actually price architectural trust commitments into purchase decisions at a level sufficient to sustain Proposition 2’s predicted revenue model, and whether GPLv3 versus MIT license framing affects perceived credibility (Proposition 3), assumptions this paper’s business-model argument relies on but does not test.

9. Conclusion

Hold-up in connected hardware is usually treated as a policy problem or an engineering problem. This paper has argued it is also a business-model design problem, addressable through the joint choice of admission and runtime openness, made credible by which mechanisms are structurally irreversible rather than by what the vendor promises. The case examined here shows that occupying the open-open quadrant is a genuine commercial trade-off—forefeiting two conventional monetisation levers for a relocated, margin-constrained revenue stream—not a costless virtue, and the paper has tried to state the trade-off honestly rather than minimise it.

What the case also demonstrates, at the level of design, is that the trade-off buys something specific: local, encrypted, always-available device control, matching what a governed platform standard’s architecture offers, plus a further data-sovereignty guarantee that architecture alone does not provide—the owner’s data remaining in the owner’s own hands, at their own disposal, never held by a party the owner cannot verify or exit. That is achieved without asking the owner to trust the vendor’s cloud or the manufacturer to pay a consortium for permission to compete. Whether the combination proves commercially sustainable at scale is a question a single, pre-launch case cannot yet answer. But the mechanisms through which it is designed to work, and the grounding in hold-up theory, are coherent, and worth making legible—for other firms evaluating similar terrain, and for policy and research-funding audiences deciding which architectures merit support as the smart-home industry’s governance choices continue to take shape.

Version 4.2 of this paper added verifiable artifacts—Figures 1–2, the Appendix A status register, an explicit license architecture as a varying-strength hostage, and a first-pass CRA alignment analysis—to move the argument from conceptual to checkable. Version 4.3 carried that same discipline into the CRA analysis itself: the product-classification claim in Section 7.1(3) is corrected against the Regulation’s actual Annex III text rather than left as an assertion, and the mapping between specific architectural mechanisms and specific CRA requirements in Section 7.1(1)–(2) is made explicit rather than implied. Version 4.6 extended the same discipline to two risk categories the earlier versions left implicit: the geopolitical fragility of shared white-label cloud platforms (Section 6.5), and the chip-level supply-chain dependency inherent in any commodity SoC, illustrated with a documented case and paired with an explicit argument that runtime sovereignty improves detectability of such compromises even where it cannot prevent them (Sections 6.1 and 8); it also situated the case against a second domain, secure messaging, in which the open-open quadrant is already commercially realised rather than merely proposed (Section 7). Version 4.7 turns the same scrutiny on the paper’s own theoretical machinery: AI-assisted development lowers the cost of exercising the GPLv3 fork threat, which Proposition 3 predicts strengthens that hostage’s credibility (Section 6.3), but the same cost reduction lowers the cost of unrelated market entry generally, which tightens the margin constraint Proposition 2 already accepts (Section 7); the paper reports both directions rather than the favourable one alone. The credible-commitment devices examined—a root key the vendor never holds (a GPLv3 hostage), a protocol published under MIT where evolution can be checked, a support relationship the customer can leave without losing the product—are not incidental features layered onto an otherwise conventional business. They are,

on the argument developed here, the business model's actual substitute for the two monetisation levers it deliberately forgoes.

References

- BleepingComputer. (2025, March 10). *Undocumented commands found in Bluetooth chip used by a billion devices*. <https://www.bleepingcomputer.com/news/security/undocumented-command-s-found-in-bluetooth-chip-used-by-a-billion-devices/>
- Boudreau, K. J. (2010). Open platform strategies and innovation: Granting access vs. devolving control. *Management Science*, 56(10), 1849–1872. <https://doi.org/10.1287/mnsc.1100.1215>
- Cimpanu, C. (2017, August 12). *Botched firmware update bricks hundreds of smart door locks*. BleepingComputer. <https://www.bleepingcomputer.com/news/hardware/botched-firmware-update-bricks-hundreds-of-smart-door-locks/>
- Connectivity Standards Alliance. (n.d.). *Become a member*. Retrieved July 2026, from <https://csa-iot.org/become-member/>
- Espressif Systems. (2025, March 10). *Response to recent Bluetooth security research findings on ESP32* [Statement]. https://www.espressif.com/en/news/Response_ESP32_Bluetooth
- European Commission. (n.d.). *Cyber Resilience Act—Summary of the legislative text*. Shaping Europe’s Digital Future. Retrieved July 2026, from <https://digital-strategy.ec.europa.eu/en/policies/cra-summary>
- European Union. (2024). *Regulation (EU) 2024/2847 of the European Parliament and of the Council of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union. Came into force 10 December 2024; main obligations apply from 11 December 2027. Annex III lists “important products with digital elements” in Class I and Class II tiers; smart-home products with security functionalities, including smart door locks, are enumerated in that Annex.
- Federal Trade Commission. (2016, July 7). *Letter from Mary K. Engle to Richard J. Lutton, Jr. re: Nest Labs, Inc.* (FTC File No. 162-3119) [Closing letter]. https://www.ftc.gov/system/files/documents/closing_letters/nid/160707nestrevolvletter.pdf
- Federal Trade Commission. (2023, May 31). *FTC says Ring employees illegally surveilled customers, failed to stop hackers from taking control of users’ cameras* [Press release]. <https://www.ftc.gov/news-events/news/press-releases/2023/05/ftc-says-ring-employees-illegally-surveilled-customers-failed-stop-hackers-taking-control-users>
- Greenwald, G. (2014). *No place to hide: Edward Snowden, the NSA, and the U.S. surveillance state*. Metropolitan Books.
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Klein, B., Crawford, R. G., & Alchian, A. A. (1978). Vertical integration, appropriable rents, and the competitive contracting process. *Journal of Law and Economics*, 21(2), 297–326. <https://doi.org/10.1086/466922>
- Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). Local-first software: You own your data, in spite of the cloud. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! ’19)* (pp. 154–178). Association for Computing Machinery. <https://doi.org/10.1145/3359591.3359737>

- Pillsbury Winthrop Shaw Pittman LLP. (2025, April 15). *The EU's Cyber Resilience Act: New cybersecurity requirements for connected products and software*. <https://www.pillsburylaw.com/en/news-and-insights/eu-cyber-resilience-act-requirements-products-software.html>
- Svensson-Höglund, S., Richter, J. L., Maitre-Ekern, E., Russell, J. D., Pihlajarinne, T., & Dalhammar, C. (2021). Barriers, enablers and market governance: A review of the policy landscape for repair of consumer electronics in the EU and the U.S. *Journal of Cleaner Production*, 288, Article 125488. <https://doi.org/10.1016/j.jclepro.2020.125488>
- Tuya Inc. (2026). *Form 20-F annual report* [Annual report]. U.S. Securities and Exchange Commission. <https://ir.tuya.com/node/7011/html>
- Tuya Inc. (n.d.). *Company overview* [Partner profile]. Alibaba Cloud. Retrieved August 2026, from <https://www.alibabacloud.com/en/partner/tuyasmart>
- VOA News. (2021, August 21). *Cybersecurity experts worried by Chinese firm's control of smart devices*. Voice of America. https://www.voanews.com/a/east-asia-pacific_voa-news-china_cybersecurity-experts-worried-chinese-firms-control-smart-devices/6209815.html
- West, J., & Gallagher, S. (2006). Challenges of open innovation: The paradox of firm investment in open-source software. *R&D Management*, 36(3), 319–331. <https://doi.org/10.1111/j.1467-9310.2006.00436.x>
- Williamson, O. E. (1979). Transaction-cost economics: The governance of contractual relations. *Journal of Law and Economics*, 22(2), 233–261. <https://doi.org/10.1086/466942>
- Williamson, O. E. (1983). Credible commitments: Using hostages to support exchange. *American Economic Review*, 73(4), 519–540.
- Yin, R. K. (2018). *Case study research and applications: Design and methods* (6th ed.). SAGE.

Licenses Referenced

- Free Software Foundation. (2007). *GNU General Public License v3.0*. Strong copyleft licence with an anti-tivoization clause relevant to secure-boot closure prevention. <https://www.gnu.org/licenses/gpl-3.0.html>
- Open Source Initiative. (n.d.). *The MIT License*. Permissive licence; weak hostage but maximizes admission openness. <https://opensource.org/license/mit>
- CERN. (2020). *CERN Open Hardware Licence Version 2—Strongly Reciprocal (CERN-OHL-S v2)*. <https://ohwr.org/project/cernohl/wikis/Documents/CERN-OHL-version-2>

Appendix A: Implementation-Status Register

This register distinguishes *specified*, *built-but-bench-verified*, and *field-verified* elements of the system. It is dated and intended as a checkable artifact, not a marketing claim. Status as of July 2026. This is what converts Figure 2's target-versus-current distinction from conceptual to auditable.

Table 5: Implementation-Status Register (as of July 2026)

Component	Description	Status		Date	Evidence / Artifact	Hostage Axis
Bridge command relay	Bridge relays encrypted command + signed token, lock ↔ broker	Built, verified	bench-	2025-11	Bench logs, MQTT capture, source in repo /bridge	Runtime
Local commissioning	BLE / local mesh unlock without internet	Built, verified	bench-	2025-10	Test rig video, mobile app logs	Runtime
Mesh networking	Bridge-to-lock mesh, offline token verification	Built, verified	bench-	2026-01	Lock verifies token offline against stored public key	Runtime
Broker pairing storage	Broker stores only the pairing relation, no log	Built, verified (plaintext fallback still present)	bench-	2026-03	DB schema review, broker code /broker	Runtime
E2E payload encryption	ECDH key exchange + AES-256-GCM, zero-knowledge broker	Specified, not implemented		2026-02 (spec)	Spec doc v0.8, target architecture (Fig. 2); plaintext fallback still in production	Runtime — strong hostage once shipped
Plaintext credential fallback removal	Remove the current plaintext credential cache on the server	Specified, not implemented		2026-03	Issue #142, scheduled pre-launch	Runtime
Owner root key generation	Root private key generated on owner's phone secure enclave, never transmitted	Built, verified	bench-	2026-04	App code /mobile/keygen; Android Keystore / iOS Secure Enclave	Runtime — core hostage
Secondary token signing + offline verify	Owner signs secondary tokens; lock verifies offline	Built, verified	bench-	2026-04	Token format spec; lock firmware verify() function	Runtime
Key recovery / backup	Recovery path when the root-credential device is lost	Open question	design	2026-07	Candidate: Shamir 2-of-3 backup + destructive repairing via physical button	Runtime
Per-device crypto identity	Identity provisioned at production; a clone fails enrolment	Specified, partial build		2026-02	Provisioning CA sketch, factory-flow document	Admission + Runtime

Continued on next page

Table 5 (continued)

Component	Description	Status	Date	Evidence / Artifact	Hostage Axis
Secure boot	Left unconfigured on consumer/hospitality tier; enabled and vendor-signed on defence tier	Implemented as designed	2026-05	eFuse config logs; defence-tier build flag SECURE_BOOT=1	Admission — hostage against closure
Server + bridge + app source publication	Publish under GPLv3	Partial — internal repo, not yet public	2026-06	Repo private; licence headers added; public release blocked pending E2E implementation	Admission — strong hostage
Protocol spec + SDK publication	Publish under MIT	Draft published internally	2026-06	Spec v0.7 in /docs/protocol; MIT header	Admission — weak hostage, maximises adoption
Hardware design	Schematics under CERN-OHL-S v2	Specified	2026-01	KiCad files; not yet fabricated	Admission
Self-host option	Owner can point the bridge to their own broker	Built, bench-verified	2026-03	Config file BRIDGE_BROKER_URL env var; tested with Mosquitto	Runtime + Admission — exit
HomeKit Bridge integration	Exposed via Home Assistant HAP non-commercial track	Built, bench-verified	2026-05	Home Assistant config; Home app screenshot	Admission — additive, not structural
Support-tier business model	Tiered technical support as revenue	Specified	2026-06	Business-model document v3; pricing draft	P2 viability

Notes. “Built, bench-verified” means the component works on the bench rig but is not yet field-verified at scale. “Specified” means a design document exists but no working code does. The plaintext fallback is the critical path to the target zero-knowledge broker shown in Figure 2. Key recovery remains open per Section 8. The register itself is intended as a Williamson hostage: publishing dated status makes reversal—for example, quietly re-enabling secure boot on the consumer tier—independently verifiable.